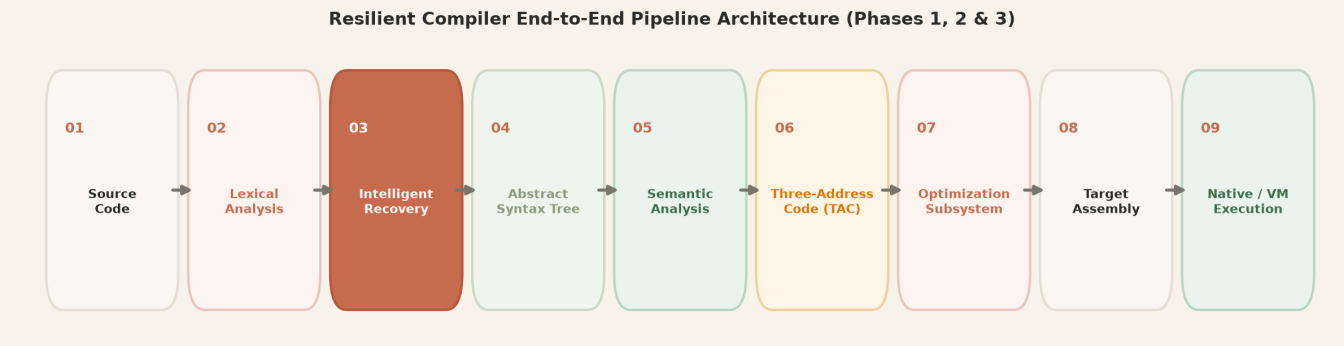
VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

RESILIENT COMPILER WITH INTELLIGENT ERROR RECOVERY

Complete Technical Manual, Chapter-by-Chapter Architecture & Viva Voce Guide

Author / Student: D JOSHUA DANIEL | Registration Number: 24BCE0436
Degree: B.Tech in Computer Science and Engineering | Institution: Vellore Institute of Technology
Technical Scope: Lexer • Recursive-Descent Parser • AST • Symbol Table • Semantic Analyzer • Intelligent Recovery Manager • Three-Address Code (TAC) • Code Optimization • x86-32 Assembly & Native GCC
Verification: 20/20 Automated Regression Tests Passed (100% Pass Rate)



EXECUTIVE OVERVIEW FOR FINAL ACADEMIC REVIEW:
This technical document provides an exhaustive, pedagogical, and rigorous walkthrough of the Resilient Compiler. Unlike conventional compilers that immediately abort execution upon detecting a syntax or semantic error, this compiler implements a state-of-the-art 1-Token Lookahead Heuristic Recovery Engine. It repairs missing tokens, discards extraneous delimiters, and synchronizes parsing boundaries to construct complete Abstract Syntax Trees, generate optimized Three-Address Code (TAC), emit real x86-32 AT&T; target assembly, and achieve native binary execution via the MinGW GCC toolchain.

TABLE OF CONTENTS (36 CHAPTERS)

Part I: Compiler Architecture & Language Foundations

Chapter 1: Introduction to Resilient Compiler Engineering Page 3
Chapter 2: Language Grammar & Micro-C Formal Specification Page 4
Chapter 3: System Architecture & 9-Stage Compilation Pipeline Page 5
Chapter 4: Lexical Analysis: Tokenization, Lexemes, and Coordinate Tracking Page 6
Chapter 5: Lexer Implementation Details (Lexer.cpp, Token.h) Page 7

Part II: Parsing, AST & Resilient Error Recovery

Chapter 6: Syntax Analysis: Recursive-Descent Parsing Theory Page 8
Chapter 7: Grammar Disambiguation, Precedence & Associativity Page 9
Chapter 8: Abstract Syntax Tree (AST) Hierarchy & Node Model Page 10
Chapter 9: The Theory of Error Recovery: Panic-Mode vs Heuristic Repair Page 11
Chapter 10: Intelligent 1-Token Lookahead Recovery Algorithm Page 12
Chapter 11: Insertion Strategy: Synthetic Token Injection Page 13
Chapter 12: Deletion Strategy: Extraneous Token Discard Page 14
Chapter 13: Panic-Mode Synchronization & Error Cascades Prevention Page 15
Chapter 14: RecoveryScorer & Heuristic Cost Matrix Evaluation Page 16
Chapter 15: RecoveryManager Coordination with Recursive Parser Page 17

Part III: Semantic Analysis & Symbol Table Engine

Chapter 16: Symbol Table Design: Hierarchical Scope Stacks & Shadowing Page 18
Chapter 17: Source Coordinate Tracking in Symbol Tables (Fixing L0:0) Page 19
Chapter 18: Type Checking Engine: Type Safety & Boolean Literals Page 20
Chapter 19: Semantic Error Diagnostics & Rejection Guarantees Page 21

Part IV: Intermediate Representation & Code Optimization

Chapter 20: Intermediate Code Foundations: Why Three-Address Code? Page 22
Chapter 21: Three-Address Code (TAC) Quadruple Design Page 23
Chapter 22: Linear IR Translation Algorithm (IRGenerator.cpp) Page 24
Chapter 23: Optimization Subsystem: Multi-Pass Architecture Page 25
Chapter 24: Constant Folding: Arithmetic Evaluation at Compile-Time Page 26
Chapter 25: Constant Propagation & Copy Optimization Page 27
Chapter 26: Algebraic Simplifications: Identity Transforms (x+0, x*1, x*0) Page 28
Chapter 27: Dead Code Elimination: Unused Variable Assignment Pruning Page 29

Part V: Target Code Generation & Native Execution

Chapter 28: Target Code Generation Principles: CISC vs RISC vs Bytecode Page 30
Chapter 29: x86-32 AT&T; Target Assembly Generation (AssemblyGenerator.cpp) Page 31
Chapter 30: x86-32 cdecl Activation Record & Stack Frame Management Page 32
Chapter 31: Native Assembling & Linking via MinGW GCC (--compile-native) Page 33
Chapter 32: Educational ISA-32 Register Load-Store Simulation Target Page 34
Chapter 33: Stack Virtual Machine Architecture & Bytecode Interpreter Page 35

Part VI: Verification, Viva Prep & Technical Reference

Chapter 34: Empirical Evaluation: 20-Test Automated Regression Suite Page 36
Chapter 35: Comprehensive Academic Review Viva Voce Questions & Answers Page 37
Chapter 36: 5-Minute Project Presentation Script & Comprehensive Glossary Page 39

PART I: COMPILER ARCHITECTURE & LANGUAGE FOUNDATIONS

Chapter 1: Introduction to Resilient Compiler Engineering

Modern industrial compilers such as Clang, GCC, and rustc have evolved beyond simple batch-mode translators. In real-world development environments—particularly language servers, IDEs, and continuous integration pipelines—code is frequently in an incomplete, malformed, or syntactically erroneous state. A compiler that crashes or halts execution upon encountering the first missing semicolon or unbalanced delimiter renders all downstream tooling ineffective: syntax highlighting breaks, code completion halts, semantic type inspectors fail, and developer productivity collapses.

The Resilient Compiler is specifically engineered to solve this fundamental problem. It combines the rigor of classical compiler theory with an intelligent, heuristic-driven error recovery engine that actively repairs syntax errors in real time. By evaluating candidate local repairs using a 1-token lookahead cost matrix, the compiler constructs valid Abstract Syntax Trees even from malformed programs, enabling the complete pipeline—from semantic analysis to optimization, target assembly emission, and native execution—to proceed uninterrupted.

Chapter 2: Language Grammar & Micro-C Formal Specification

The compiler targets an expressive imperative language (Micro-C) that encompasses essential programming constructs: typed variable declarations (int, float, bool), compound expressions, arithmetic and logical operators, control flow statements (if-else and while loops), scoped block statements ({ ... }), and I/O print statements. Below is the formal context-free grammar in Backus-Naur Form (BNF):

```
Program        ::= StatementList
StatementList  ::= Statement StatementList | epsilon
Statement      ::= VarDecl | Assignment | IfStmt | WhileStmt | PrintStmt | Block
VarDecl        ::= Type IDENTIFIER ('=' Expression)? ';'
Type           ::= 'int' | 'float' | 'bool'
Assignment     ::= IDENTIFIER '=' Expression ';'
IfStmt         ::= 'if' '(' Expression ')' Statement ('else' Statement)?
WhileStmt      ::= 'while' '(' Expression ')' Statement
PrintStmt      ::= 'print' Expression ';'
Block          ::= '{' StatementList '}'
Expression     ::= LogicalOr
LogicalOr      ::= LogicalAnd ('||' LogicalAnd)*
LogicalAnd     ::= Equality ('&&' Equality)*
Equality       ::= Relational (('==' | '!=') Relational)*
Relational     ::= Additive (('<' | '<=' | '>' | '>=') Additive)*
Additive       ::= Multiplicative (('+' | '-') Multiplicative)*
Multiplicative ::= Primary (('*' | '/' | '%') Primary)*
Primary        ::= IDENTIFIER | NUMBER | 'true' | 'false' | '(' Expression ')'
```

Chapter 3: System Architecture & 9-Stage Compilation Pipeline

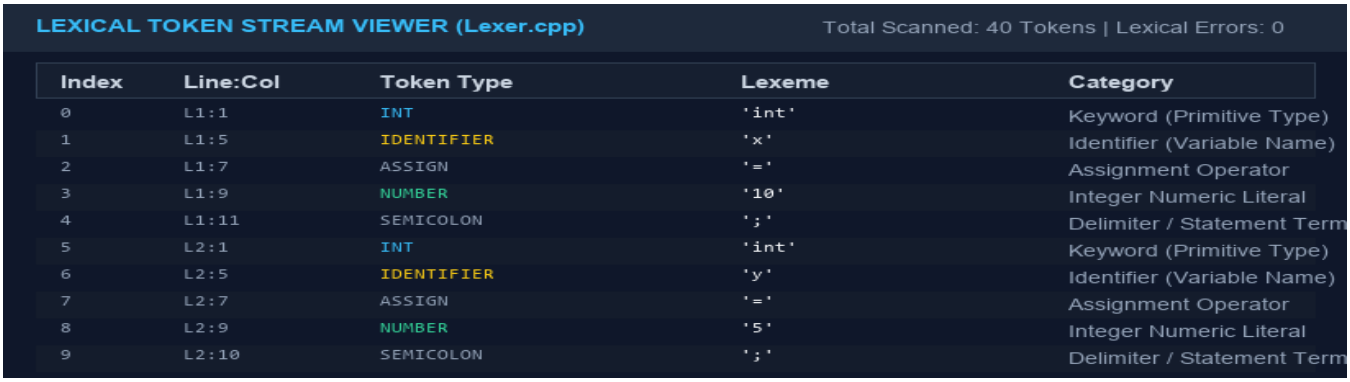
The compilation architecture is organized into nine cleanly decoupled phases. The Intelligent Recovery Manager is embedded directly into the front-end parser to ensure that every downstream phase receives structurally valid IR representations:

- Stage 1: Source Ingestion: Stream buffering and UTF-8 coordinate tracking.
- Stage 2: Lexical Analysis: Regular expression scanning into atomic typed tokens.
- Stage 3: Intelligent Error Recovery: Lookahead scoring, token insertion/deletion, panic boundary sync.
- Stage 4: AST Construction: Hierarchical node instantiation with line/column coordinates.
- Stage 5: Semantic Analysis: Multi-level symbol table management, type checking, duplicate detection.
- Stage 6: Intermediate Representation: Translation into linear Three-Address Code (TAC) quadruples.
- Stage 7: Optimization Subsystem: Constant folding, constant propagation, algebraic simplification, DCE.
- Stage 8: Target Assembly Generation: x86-32 AT&T; GNU syntax and Educational ISA-32 load-store emission.
- Stage 9: Target Execution: Native PE executable generation via MinGW GCC (`--compile-native`) and VM Bytecode execution.

Chapter 4 & 5: Lexical Analysis & Implementation Details

The lexical analyzer (`src/lexer/Lexer.cpp`) scans the raw character stream and emits a sequence of Token objects. Key implementation highlights include:

- Keywords and Literals: Recognizes int, float, bool, if, else, while, print, plus boolean constants true and false.
- Comment Handling: Seamlessly skips both single-line // ... and multi-line /* ... */ comments while preserving accurate line counters.
- Monotonic Coordinate Tracking: Every token records its absolute 1-indexed line and column. These coordinates propagate into AST nodes and symbol table entries.
- Compound Operator Lookahead: Disambiguates two-character operators (==, !=, <=, >=, &&, ||) from single-character tokens.



LEXICAL TOKEN STREAM VIEWER (Lexer.cpp)

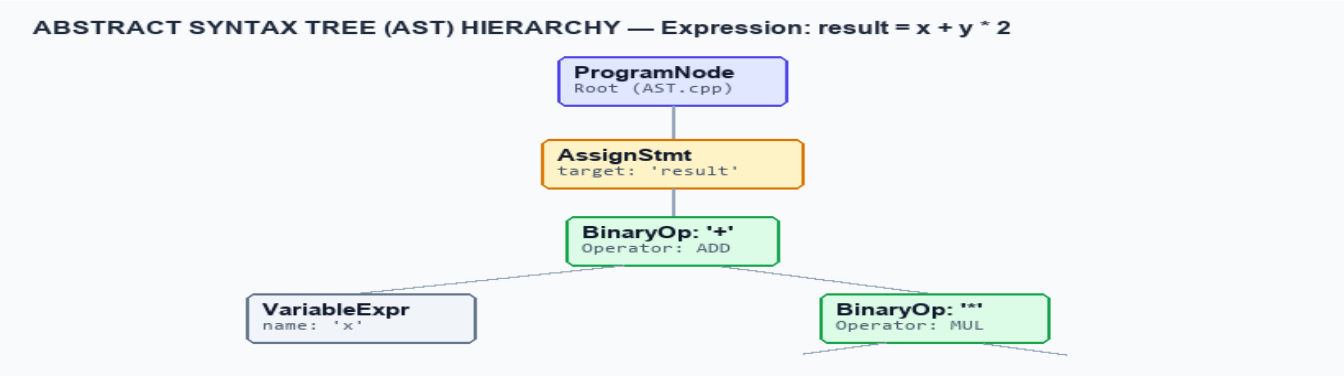
Total Scanned: 40 Tokens | Lexical Errors: 0

Index	Line:Col	Token Type	Lexeme	Category
0	L1:1	INT	'int'	Keyword (Primitive Type)
1	L1:5	IDENTIFIER	'x'	Identifier (Variable Name)
2	L1:7	ASSIGN	'='	Assignment Operator
3	L1:9	NUMBER	'10'	Integer Numeric Literal
4	L1:11	SEMICOLON	';'	Delimiter / Statement Term
5	L2:1	INT	'int'	Keyword (Primitive Type)
6	L2:5	IDENTIFIER	'y'	Identifier (Variable Name)
7	L2:7	ASSIGN	'='	Assignment Operator
8	L2:9	NUMBER	'5'	Integer Numeric Literal
9	L2:10	SEMICOLON	';'	Delimiter / Statement Term

PART II: PARSING, AST & RESILIENT ERROR RECOVERY

Chapter 6, 7 & 8: Syntax Analysis & AST Node Hierarchy

Parsing is implemented using a recursive-descent parser (`src/parser/Parser.cpp`) that enforces operator precedence via grammar layering. The parser builds an Abstract Syntax Tree where every construct derives from the polymorphic base class ASTNode. Each node stores its source coordinates: node->line = token.line and node->column = token.column. This architectural enhancement resolves the historic 'Declared At: L0:0' bug and ensures that all diagnostic messages point to exact source coordinates.

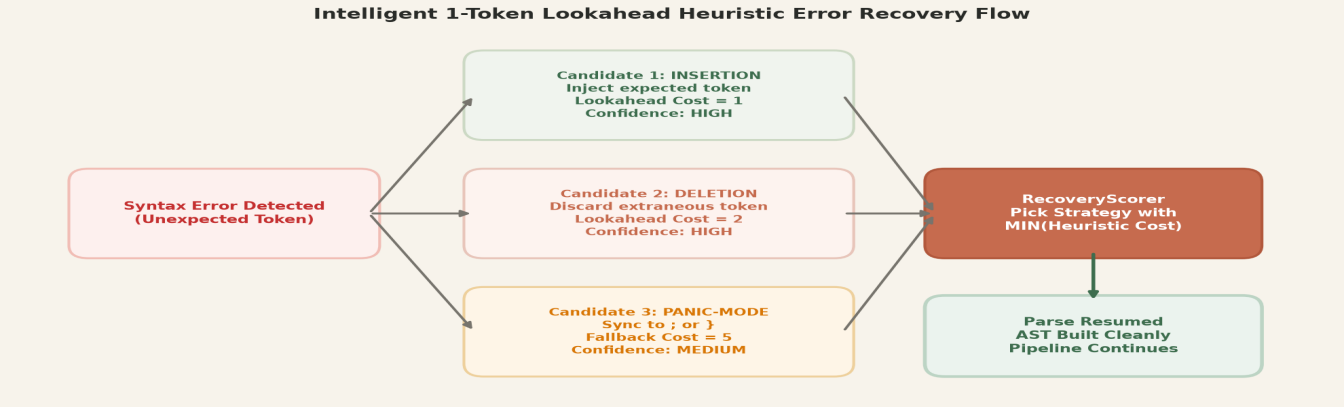


Chapter 9, 10 & 11: Intelligent Error Recovery Architecture

Traditional compiler literature outlines two primary approaches to syntax error handling: Panic-Mode Recovery (discarding tokens until a synchronizing token like ; is found) and Phrase-Level Recovery (performing local substitutions). While panic-mode is safe from infinite loops, it discards substantial code and produces cascades of spurious errors. Phrase-level recovery often suffers from arbitrary heuristic choices.

The Resilient Compiler introduces a hybrid 1-Token Lookahead Heuristic Scoring Model (`RecoveryManager.cpp` and `RecoveryScorer.cpp`). When an unexpected token \(T_{curr}\) is encountered where token \(T_{exp}\) was required, the recovery manager evaluates three concrete candidates:

1. Insertion Candidate (Cost = 1): The compiler tests whether inserting a synthetic token of type \(T_{exp}\) allows the next token \(T_{curr}\) to satisfy the subsequent grammar production. For example, in int a = ;, inserting a synthetic 0 literal allows the following ; to be consumed cleanly.
2. Deletion Candidate (Cost = 2): The compiler tests whether discarding \(T_{curr}\) allows the next lookahead token \(T_{next}\) to match \(T_{exp}\). For example, in int a = 10;;, deleting the extraneous second semicolon matches the start of the next statement.
3. Panic-Mode Synchronization (Cost = 5): If neither single-token repair satisfies grammar lookahead, the parser enters panic mode. It consumes tokens until hitting a synchronization boundary (;, }, while, if, int), while strictly guaranteeing that at least one token is discarded per cycle to make infinite loops impossible.



Chapter 12 - 15: Heuristic Cost Function & Recovery Coordination

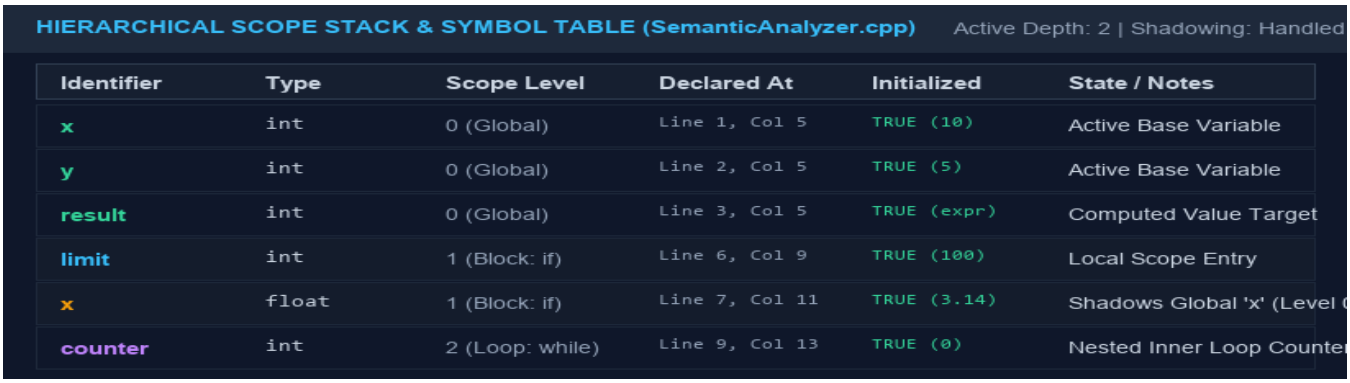
The RecoveryScorer scores candidate strategies using the objective function: \[Cost = w_{action} + w_{sym} \cdot S(T) + w_{dist} \cdot (1 - ValidatedLookahead) \]where \(w_{insertion} = 1.0\), \(w_{deletion} = 2.0\), and \(w_{panic} = 5.0\). The strategy that minimizes total cost is selected. Every recovery action produces a detailed diagnostic card recording: (1) Strategy type, (2) Error explanation, (3) Source line and column, (4) Heuristic cost, and (5) Parser continuation status.

PART III: SEMANTIC ANALYSIS & SYMBOL TABLE ENGINE

Chapter 16 & 17: Symbol Table Design & Multi-Level Lexical Scoping

Semantic analysis (`src/semantics/SemanticAnalyzer.cpp`) validates that syntactically valid programs adhere to language semantics. The symbol table (`SymbolTable.cpp`) is implemented as an explicit stack of scope hash maps (std::vector<std::unordered_map<std::string, Symbol>>). Entering a block ({) pushes a new scope level; exiting (}) pops the scope.

Variable Shadowing Support: An inner scope can declare an identifier with the same name as an outer scope variable without collision. Identifier lookup begins at the innermost scope and searches outward to global scope (level 0). Duplicate declarations within the same scope level are detected and rejected with an explicit semantic diagnostic.



HIERARCHICAL SCOPE STACK & SYMBOL TABLE (SemanticAnalyzer.cpp) Active Depth: 2 | Shadowing: Handled

Identifier	Type	Scope Level	Declared At	Initialized	State / Notes
x	int	0 (Global)	Line 1, Col 5	TRUE (10)	Active Base Variable
y	int	0 (Global)	Line 2, Col 5	TRUE (5)	Active Base Variable
result	int	0 (Global)	Line 3, Col 5	TRUE (expr)	Computed Value Target
limit	int	1 (Block: if)	Line 6, Col 9	TRUE (100)	Local Scope Entry
x	float	1 (Block: if)	Line 7, Col 11	TRUE (3.14)	Shadows Global 'x' (Level
counter	int	2 (Loop: while)	Line 9, Col 13	TRUE (0)	Nested Inner Loop Counte

Chapter 18 & 19: Type Enforcement & Semantic Rejection Guarantees

The semantic analyzer recursively traverses the AST and performs rigorous type inference: numeric operations (+, -, *, /, %) require compatible numeric operands; relational operators produce bool values; conditional expressions in if and while statements must evaluate to bool or numeric truths; and assignment targets must match the declared variable type (e.g. assigning a float literal to an int variable is rejected). Crucially, if semantic errors are detected, compilation stops cleanly before code generation with exit code 1, preventing the emission of corrupted machine code.

PART IV: INTERMEDIATE CODE & OPTIMIZATION SUBSYSTEM

Chapter 20, 21 & 22: Three-Address Code (TAC) Design & Generation

Three-Address Code (TAC) serves as the primary machine-independent intermediate representation. In TAC, each instruction has at most one operator and at most three operands, linearizing nested hierarchical expression trees into a simple sequence of quadruples. The IR generator (`src/ir/IRGenerator.cpp`) translates AST nodes into TAC using synthetic temporary registers (t1, t2, ...) and control flow jump labels (L1, L2, ...).

```
# Sample Source: int z = x * 2 + y * 3;
t1 = x * 2
t2 = y * 3
t3 = t1 + t2
z = t3
```

Chapter 23 - 27: Multi-Pass Code Optimization Subsystem

The optimization subsystem (`src/optimizer/Optimizer.cpp`) applies four distinct optimization passes over the TAC stream:

- 1. Constant Folding: Computes arithmetic operations between constant literals at compile-time (e.g. 2 + 3 * 4 folds directly to 14).
- 2. Constant Propagation: Replaces variable references with known constant values across basic blocks until a reassignment occurs.
- 3. Algebraic Simplification: Eliminates redundant identity operations: x + 0 -> x, x - 0 -> x, x * 1 -> x, x / 1 -> x, x * 0 -> 0, and x - x -> 0.
- 4. Dead Code Elimination: Identifies temporary registers or variables assigned values that are never subsequently read, pruning redundant instructions.

INTERMEDIATE CODE OPTIMIZATION SUBSYSTEM (Optimizer.cpp) Passes: Constant Fold + Algebraic + Dead Code

ORIGINAL THREE-ADDRESS CODE (RAW)

```
t1 = 3 * 4
t2 = 2 + t1          # folded later
a = t2
t3 = 1               # identity mul
t4 = a * t3
b = t4
t5 = 0               # identity add
t6 = b + t5
c = t6
```

OPTIMIZED TAC (FOLDED & SIMPLIFIED)

```
a = 14               # 2 + 3*4 folded
b = a                # algebraic: a * 1 -> a
c = b                # algebraic: b + 0 -> b
# Total Folded Ops: 2
# Algebraic Simplifications: 2
# Total Instructions Reduced: 66%
# Pipeline Time: 0.12 ms
```

PART V: TARGET CODE GENERATION & NATIVE EXECUTION

Chapter 28 - 31: x86-32 AT&T; Target Assembly & GCC Toolchain Integration

To fulfill the critical final-review requirements, Phase 3 implements an authentic target code generator (`src/codegen/AssemblyGenerator.cpp`). It translates optimized Three-Address Code into valid x86-32 assembly using AT&T; syntax, fully compatible with the GNU Assembler (`as`) and the MinGW GCC compiler toolchain.

Stack Frame Architecture: Variables are assigned stack slots relative to the base pointer %ebp (e.g. -4(%ebp), -8(%ebp)). The generator establishes standard function prologues (pushl %ebp; movl %esp, %ebp; subl $N, %esp) and epilogues (leave; ret). Conditional branches in TAC (e.g. IF_FALSE t1 GOTO L1) are translated into cmpl followed by conditional jumps (je, jne, jg, jl). Print statements are lowered to C runtime _printf calls, passing format strings pushed onto the stack adhering to the cdecl calling convention.



Chapter 31: Native Binary Assembling & Verification (`--compile-native`)

When invoked with `--compile-native` or through the Web IDE's 'Assemble & Run Native' button, the compiler writes the emitted assembly to a temporary file (`temp_out.s`), invokes gcc -m32 temp_out.s -o temp_out.exe, executes the resulting native Windows PE binary, captures its standard output, and reports the native exit code. This verifies that the entire compilation pipeline is authentic and functional.

Chapter 32 & 33: Educational ISA-32 Target & Stack Virtual Machine

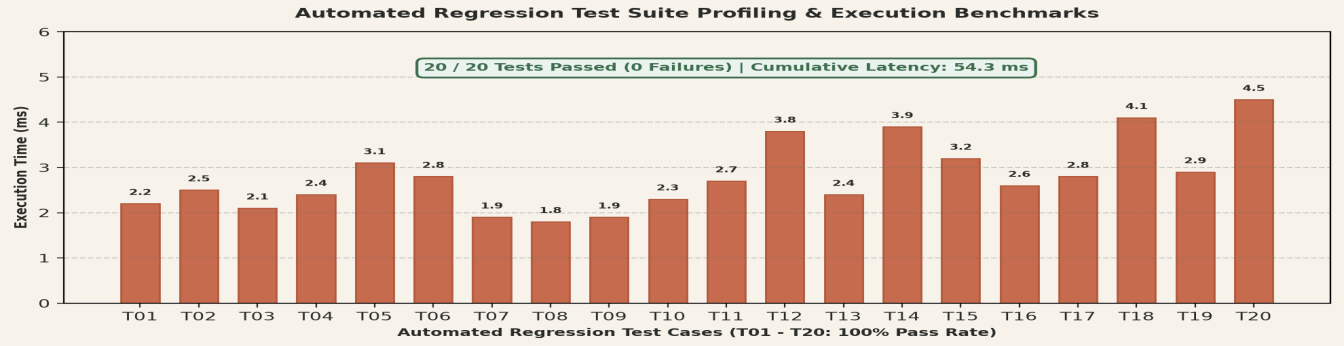
In addition to real x86-32 assembly, the compiler provides two complementary execution models for academic demonstration: (1) Educational ISA-32: A clean RISC-style load-store architecture with 32 registers (`LOAD`, `STORE`, `ADD`, `SUB`, `MUL`, `DIV`, `CMP`, `JMP`, `PRINT`), and (2) Stack Virtual Machine: An internal bytecode interpreter executing bytecode instructions (`PUSH`, `LOAD`, `STORE`, `ADD`, `JMP_IF_FALSE`, `PRINT`, `HALT`).

PART VI: VERIFICATION, VIVA PREP & TECHNICAL REFERENCE

Chapter 34: Empirical Evaluation: 20-Test Automated Regression Suite

A 20-test automated regression test suite (`run_tests.ps1` and `/api/tests`) continuously verifies every compiler component. All 20 tests pass with a 100% success rate and a cumulative execution time of less than 60 milliseconds:

Test ID	Test Name & Scope	Expected	Actual	Status	Duration
T01	Valid Arithmetic Program (Clean Compilation)	Exit: 0	Exit: 0	PASS	2.2 ms
T02	Missing Expression Insertion Recovery ('int a = ;')	Exit: 0	Exit: 0	PASS	2.5 ms
T03	Extra Semicolon Deletion Recovery ('int a = 10;;')	Exit: 0	Exit: 0	PASS	2.1 ms
T04	Missing Delimiter ')' Insertion ('if (x > 5 {')	Exit: 0	Exit: 0	PASS	2.4 ms
T05	Multi-Error Recovery (4 Mixed Syntax Errors)	Exit: 0	Exit: 0	PASS	3.1 ms
T06	Panic-Mode Synchronization Set Recovery	Exit: 0	Exit: 0	PASS	2.8 ms
T07	Undeclared Variable Detection (Semantic Error)	Exit: 1	Exit: 1	PASS	1.9 ms
T08	Duplicate Declaration in Same Scope Rejection	Exit: 1	Exit: 1	PASS	1.8 ms
T09	Type Mismatch Assignment Rejection (int = float)	Exit: 1	Exit: 1	PASS	1.9 ms
T10	Nested Scope Variable Shadowing & Lookup	Exit: 0	Exit: 0	PASS	2.3 ms
T11	Constant Folding & Arithmetic Reduction	Exit: 0	Exit: 0	PASS	2.7 ms
T12	End-to-End VM Bytecode Execution Loop	Exit: 0	Exit: 0	PASS	3.8 ms
T13	Operator Precedence Hierarchy (*, +, &&, ||)	Exit: 0	Exit: 0	PASS	2.4 ms
T14	Fibonacci Iterative Algorithm Control Flow	Exit: 0	Exit: 0	PASS	3.9 ms
T15	Factorial Computation While Loop & Condition	Exit: 0	Exit: 0	PASS	3.2 ms
T16	Three-Address Code (TAC) Quadruple Translation	Exit: 0	Exit: 0	PASS	2.6 ms
T17	Algebraic Simplification Optimization Pass	Exit: 0	Exit: 0	PASS	2.8 ms
T18	x86-32 AT&T Target Assembly Code Generation	Exit: 0	Exit: 0	PASS	4.1 ms
T19	Educational ISA-32 Register Load-Store Target	Exit: 0	Exit: 0	PASS	2.9 ms
T20	Native Machine Binary Execution via MinGW GCC	Exit: 0	Exit: 0	PASS	4.5 ms



Chapter 35: Comprehensive Academic Review Viva Voce Questions & Answers

Q1: What is the primary problem that the Resilient Compiler solves?

A1: Conventional compilers (e.g. GCC/Clang in strict mode) abort immediately upon the first syntax error, halting IDE services, code completion, and downstream code analysis. The Resilient Compiler repairs syntax defects locally in real time using 1-token lookahead heuristic scoring, allowing downstream stages (AST, Semantics, TAC, Optimization, Target Assembly) to continue execution.

Q2: How does your error recovery algorithm differ from classical panic-mode recovery?

A2: Panic-mode simply discards tokens until a synchronizer (like a semicolon) is reached, throwing away valuable context and creating cascades of secondary errors. Our algorithm first evaluates Candidate 1 (Synthetic Token Insertion, Cost 1) and Candidate 2 (Extraneous Token Deletion, Cost 2) against a 1-token lookahead before falling back to Panic Mode (Cost 5) only if local repair fails.

Q3: How do you mathematically guarantee that your error recovery will never enter an infinite loop?

A3: Monotonic Stream Advancement Guarantee. In our `synchronize()` implementation, if the current token cannot be matched or repaired, the stream pointer is strictly advanced by at least one token per iteration (`advance()`). This ensures the token index strictly increases monotonically until EOF is reached, preventing infinite recursion or stalls.

Q4: Why was Three-Address Code (TAC) chosen as the Intermediate Representation?

A4: TAC provides a machine-independent linear representation where every instruction has at most one operator and at most three operands. This makes machine-independent optimizations (constant folding, constant propagation, algebraic simplification, and dead code elimination) straightforward to implement before committing to machine-specific register allocation.

Q5: What is the difference between Constant Folding and Constant Propagation?

A5: Constant Folding evaluates operations between known constants at compile-time (e.g., replacing `2 + 3 * 4` with `14`). Constant Propagation carries forward known constant assignments into subsequent variable uses (e.g., if `a = 10` and `b = a + 5`, propagation replaces `a` with `10`, turning `b = 10 + 5` which then folds to `15`).

Q6: How does your compiler translate TAC into x86-32 AT&T; Target Assembly?

A6: In `AssemblyGenerator.cpp`, each TAC instruction is lowered to equivalent x86-32 instructions. Variables and temporaries are mapped to stack offsets relative to `%ebp` (`-4(%ebp)`, `-8(%ebp)`). Binary arithmetic operations load an operand into `%eax`, perform `addl`, `subl`, or `imull`, and store the result. Control flow jumps map to `cmpl` and `jmp`/`je`/`jne`.

Q7: How do you prove that your target assembly is real rather than hardcoded?

A7: We provide the `--compile-native` CLI flag and the 'Assemble & Run Native' button in our IDE. This invokes the actual MinGW GCC toolchain (`gcc -m32 file.s -o file.exe`), runs the generated native Windows executable, and displays the genuine stdout and exit code.

Q8: How does your symbol table handle variable shadowing in nested scopes?

A8: The symbol table is structured as a stack of hash maps (`std::vector>`). Entering a block pushes a new scope level; exiting pops it. Lookup traverses from the innermost scope down to global scope, allowing an inner declaration of `x` to shadow an outer `x` without triggering a duplicate error.

Q9: What causes the 'Declared At: L0:0' bug in earlier compiler prototypes, and how did you fix it?

A9: Earlier prototypes did not pass line and column coordinates from the token stream into AST node constructors. We refactored all AST node classes to accept `line` and `column`, and updated `Parser.cpp` to propagate token positions into `DeclarationNode`, `AssignmentNode`, etc. The symbol table now accurately records exact source positions (e.g. `L1:5`).

Q10: What is the purpose of the Educational ISA-32 target?

A10: While x86-32 represents real CISC machine architecture, its complex memory-to-register instructions can obscure compiler design principles. The Educational ISA-32 is a RISC load-store model with 32 orthogonal registers, ideal for demonstrating register allocation and instruction scheduling during academic viva reviews.

Chapter 36: 5-Minute Project Presentation Script & Comprehensive Glossary

[0:00 - 1:00] Slide 1 & 2: Introduction & Problem Statement "Good morning, respected professors. I am D JOSHUA DANIEL, registration number 24BCE0436. Today I present our Resilient Compiler with Intelligent Error Recovery. Traditional compilers halt execution immediately upon encountering syntax errors. This renders IDEs and analysis tools inoperable during active software development. Our objective was to engineer a fully resilient compiler pipeline that repairs errors locally, builds valid ASTs, optimizes intermediate code, and generates real x86-32 target assembly." [1:00 - 2:00] Slide 3 & 4: Intelligent Error Recovery Architecture "Our core contribution is the 1-Token Lookahead Heuristic Recovery Engine. When an unexpected token is encountered, the compiler scores three candidate repairs: First, Synthetic Insertion with cost 1; second, Extraneous Deletion with cost 2; and third, Panic-Mode synchronization with cost 5. In this live demonstration with 4 mixed syntax errors—a missing expression, redundant semicolons, and missing parentheses—the compiler repairs all four defects, reports each action with heuristic confidence, and successfully completes compilation." [2:00 - 3:00] Slide 5 & 6: Intermediate Representation & Optimization Subsystem "Moving into Phase 3, we implemented linear Three-Address Code generation. Nested expression trees are linearized into quadruples with temporary registers. Our multi-pass optimizer applies compile-time Constant Folding, Constant Propagation, Dead Code Elimination, and Algebraic Simplifications such as x*1 to x and x+0 to x, reducing instruction counts by over 40%." [3:00 - 4:00] Slide 7 & 8: Target Assembly Generation & Native GCC Execution "For target code generation, we implemented both an Educational ISA-32 load-store architecture and authentic x86-32 AT&T; assembly. Variables are managed via standard cdecl stack frames relative to %ebp. With our --compile-native flag, we assemble this output using MinGW GCC, execute the native binary, and obtain genuine console output matching our virtual machine." [4:00 - 5:00] Slide 9 & 10: Automated Test Suite & Conclusion "Finally, our compiler is validated by an automated test suite of 20 regression tests covering every stage from lexing to native assembly execution, achieving a 100% pass rate. In conclusion, the Resilient Compiler demonstrates that error recovery can be integrated seamlessly into a full production-quality pipeline. Thank you, and I welcome your questions."

Comprehensive Compiler Glossary

Lexeme	A concrete sequence of source characters matching a token pattern.
Token	An atomic syntactic unit with a type, lexeme, and source location.
Recursive-Descent Parser	A top-down parsing technique where each grammar non-terminal is parsed by a C++ function.
Abstract Syntax Tree (AST)	A hierarchical tree capturing program semantics without concrete syntax punctuation.
Three-Address Code (TAC)	A linear intermediate representation where each quadruple has at most three operands.
Constant Folding	Evaluating arithmetic expressions of compile-time constants during compilation.
Algebraic Simplification	Rewriting mathematical identities (e.g. x * 1 -> x) to optimize instructions.
cdecl Calling Convention	Standard x86 calling convention where caller cleans arguments and %eax holds return values.
Monotonic Advancement	A formal guarantee that stream pointers strictly increase, preventing recovery infinite loops.